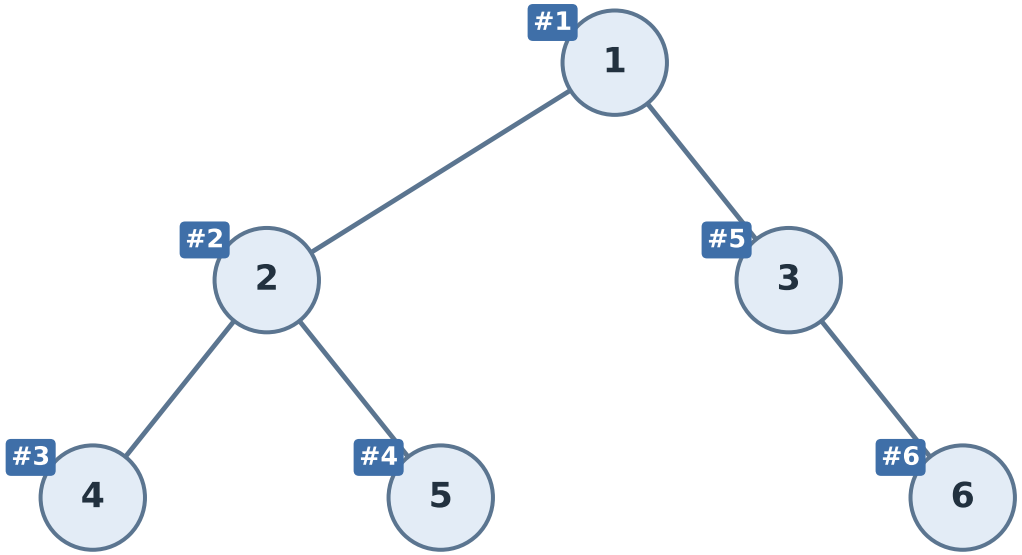
M18-01 — Build a Binary Tree and Print Preorder

Difficulty: Easy | Topic: Trees, BSTs & Heaps



Sample input

```
7
1 2 3 4 5 -1 6
```

Sample output

```
1 2 4 5 3 6
```

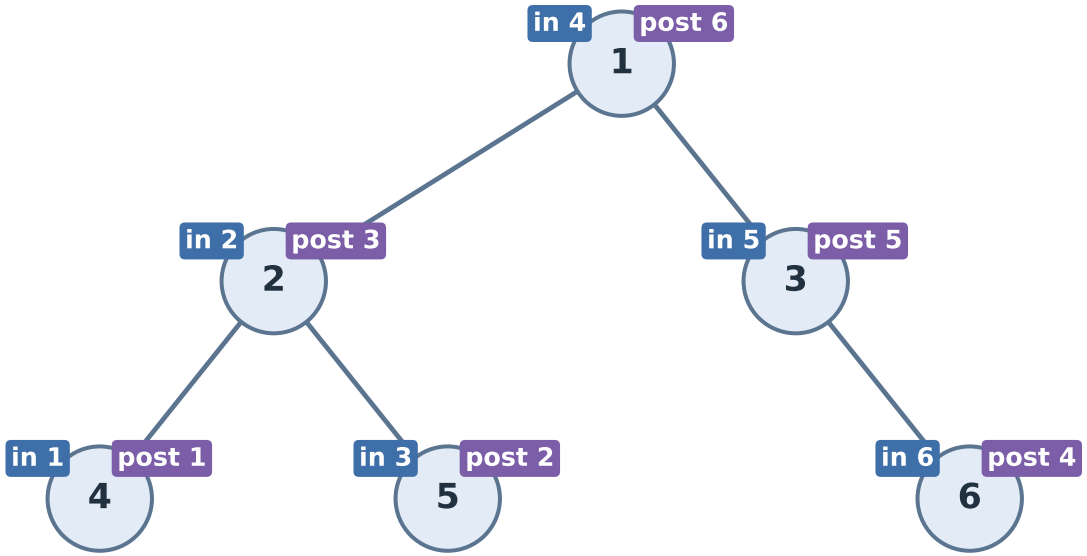
Blue badges show the order in which preorder visits each node (root, then left subtree, then right subtree).

Walkthrough

Visit 1, go left to 2 and visit its subtree (2, 4, 5), then go right to 3 and visit its subtree (3, 6). Node 3 has no left child, so 6 is its right child.

M18-02 — Print Inorder and Postorder Traversals

Difficulty: Easy | Topic: Trees, BSTs & Heaps



inorder position postorder position

Each node shows its position in the inorder traversal (blue) and the postorder traversal (purple).

Sample input

```
7
1 2 3 4 5 -1 6
```

Sample output

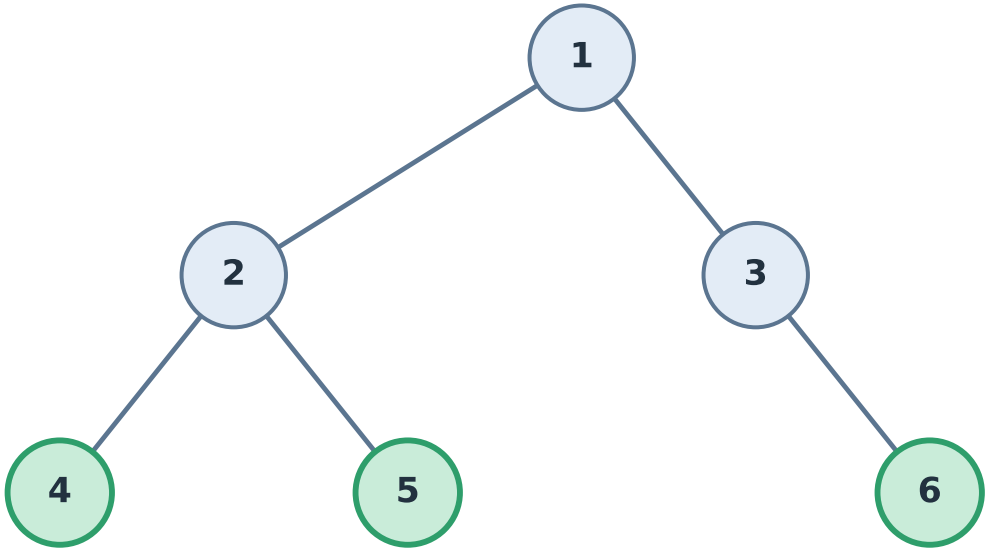
```
4 2 5 1 3 6
4 5 2 6 3 1
```

Walkthrough

Inorder visits left subtree, root, right subtree: 4 2 5 1 3 6. Postorder visits both subtrees before the root: 4 5 2 6 3 1, so the root 1 comes last.

M18-03 — Count Nodes and Leaf Nodes

Difficulty: Easy | Topic: Trees, BSTs & Heaps



leaf (no children) internal node

Green nodes are leaves. The -1 entry in the input is a missing child, not a node.

Sample input

```
7
1 2 3 4 5 -1 6
```

Sample output

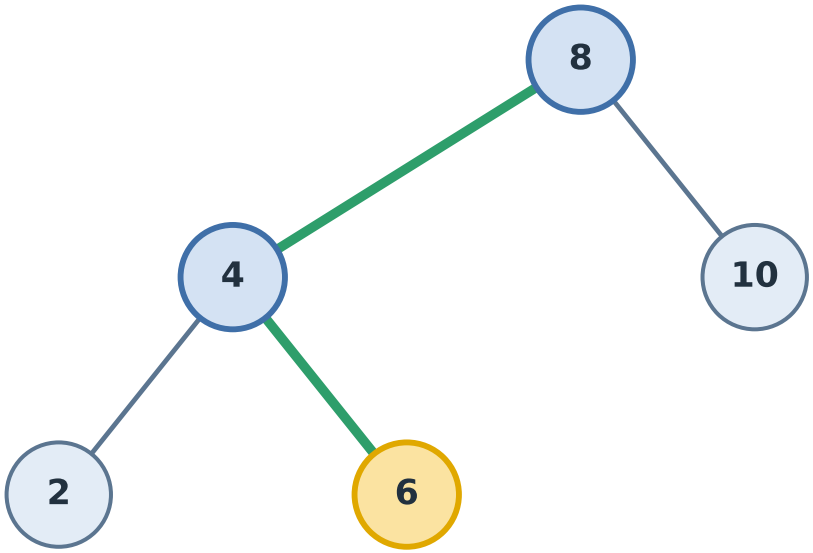
```
6 3
```

Walkthrough

There are 6 actual nodes. Nodes 4, 5, and 6 have no children, so there are 3 leaves. Node 3 has only a right child, so it is not a leaf.

M18-04 — Search in a Binary Search Tree

Difficulty: Easy | Topic: Trees, BSTs & Heaps



search path compared target found

Values are inserted in the order 8 4 10 2 6. Green edges show the search for X = 6.

Walkthrough

6 < 8, so go left to 4. 6 > 4, so go right to 6, which equals the target. The search checks 3 nodes instead of all 5.

Sample input

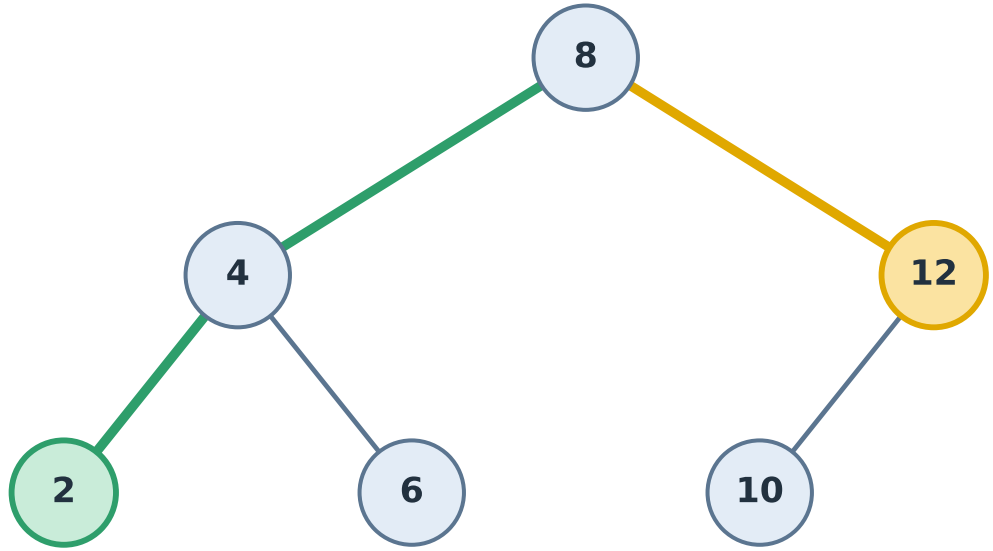
```
5 6
8 4 10 2 6
```

Sample output

```
FOUND
```

M18-05 — Find Minimum and Maximum in a BST

Difficulty: Easy | Topic: Trees, BSTs & Heaps



left links → minimum right links → maximum

Following left links from the root reaches the minimum (green); following right links reaches the maximum (gold).

Sample input

```
6
8 4 12 2 6 10
```

Sample output

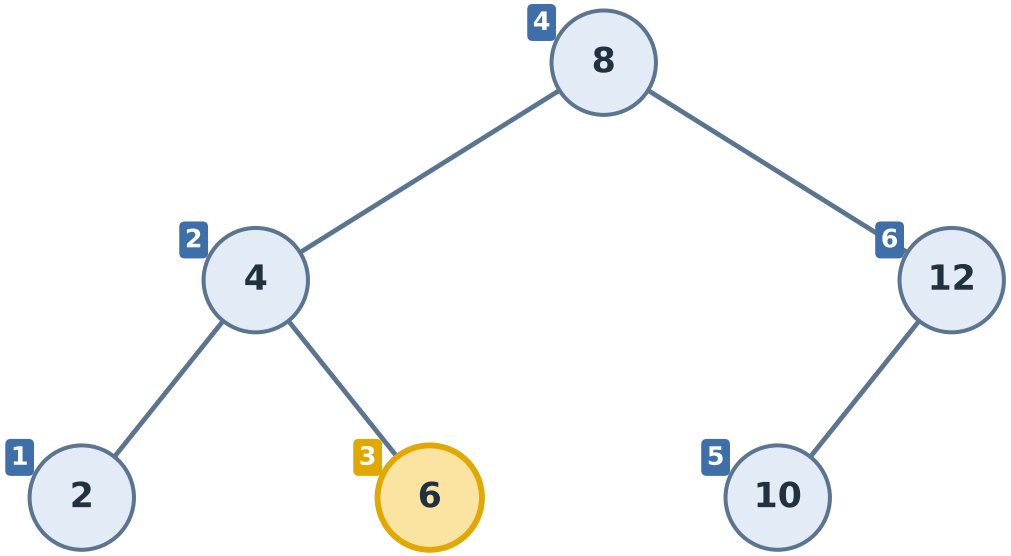
```
2 12
```

Walkthrough

From 8, go left to 4 and then to 2, which has no left child: the minimum is 2. From 8, go right to 12, which has no right child: the maximum is 12.

M18-06 — Find the Kth Smallest Value in a BST

Difficulty: Easy | Topic: Trees, BSTs & Heaps



Sample input

```
6 3
8 4 12 2 6 10
```

Sample output

```
6
```

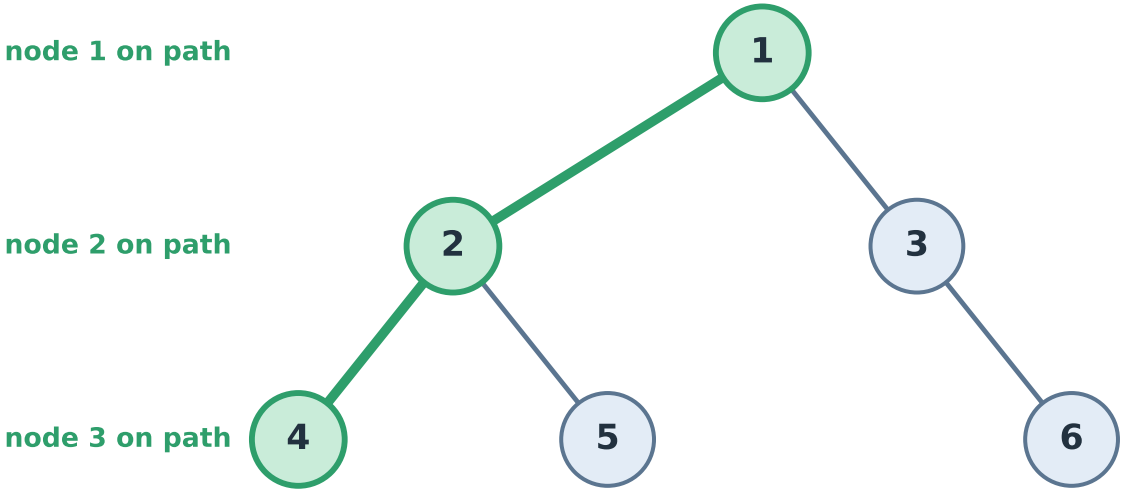
Badges show each value's rank in the inorder traversal, which visits a BST in ascending order. Gold: rank K = 3.

Walkthrough

Inorder gives 2 4 6 8 10 12. Counting visits, the traversal reaches its 3rd node at 6 and can stop there.

M18-07 — Calculate the Height of a Binary Tree

Difficulty: Medium | Topic: Trees, BSTs & Heaps



Sample input

```
7
1 2 3 4 5 -1 6
```

Sample output

```
3
```

Height counts nodes on the longest root-to-leaf path. One such path is highlighted.

Walkthrough

1 → 2 → 4 contains three nodes, so the height is 3. The paths 1 → 2 → 5 and 1 → 3 → 6 also have three nodes; none is longer.

M18-08 — Check Whether Two Binary Trees Are Identical

Difficulty: Medium | Topic: Trees, BSTs & Heaps



Sample input

```
3 3
1 2 3
1 2 4
```

Sample output

```
NO
```

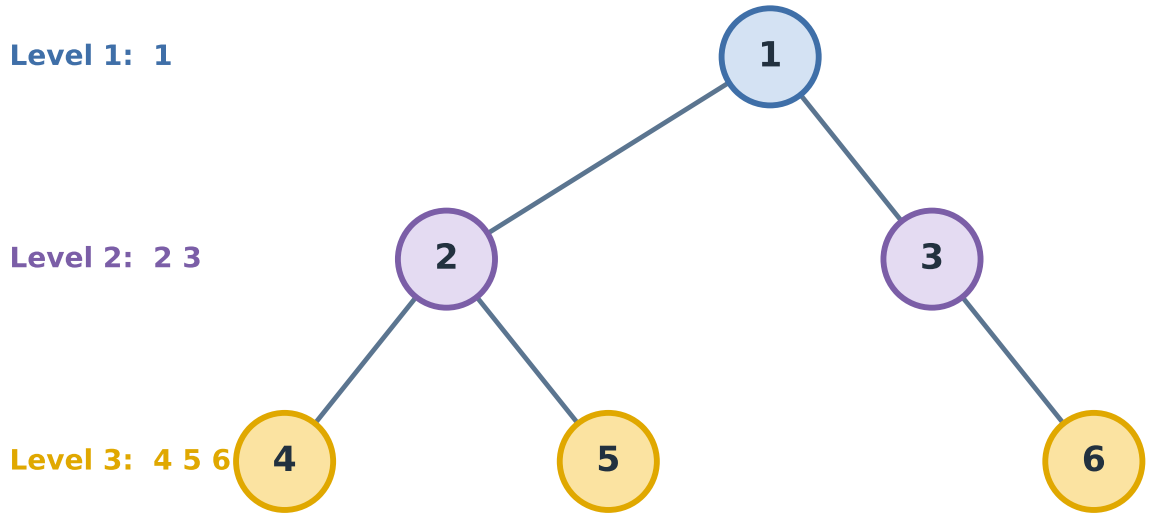
The two trees are compared position by position. Red: the corresponding nodes whose values differ.

Walkthrough

Both trees have the same shape and the same root (1) and left child (2), but the right child is 3 in tree A and 4 in tree B, so the answer is NO.

M18-09 — Level-Order Traversal by Line

Difficulty: Medium | Topic: Trees, BSTs & Heaps



Sample input

```
7
1 2 3 4 5 -1 6
```

Sample output

```
1
2 3
4 5 6
```

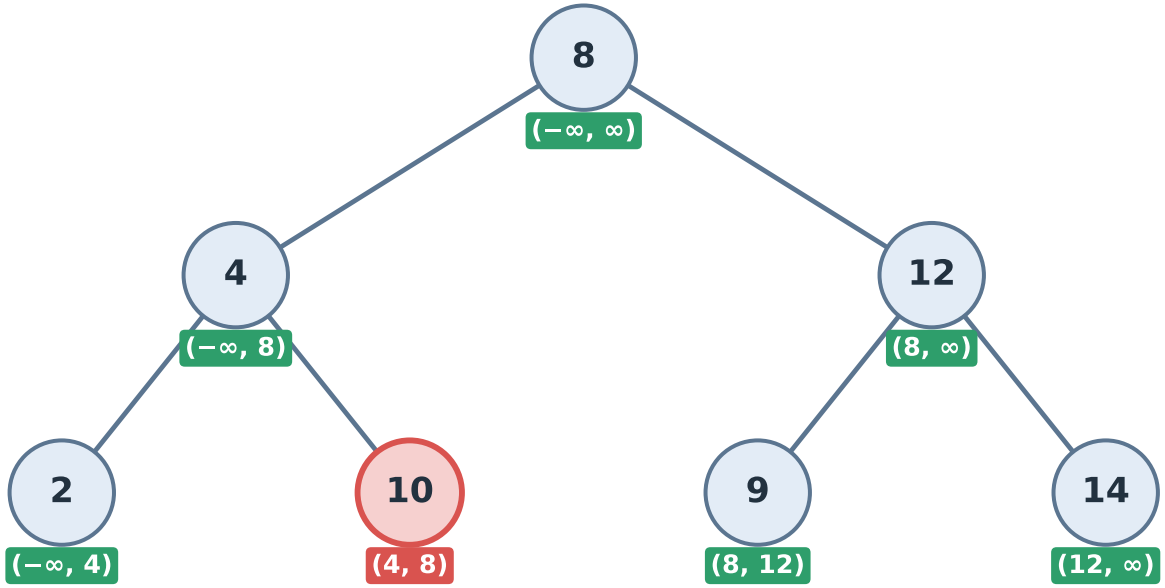
Each color is one level; BFS prints every node of a level before moving to the next.

Walkthrough

The queue starts with 1. Removing 1 adds 2 and 3 (level 2); removing them adds 4, 5, and 6 (level 3). Recording the queue size at the start of each level separates the lines.

M18-10 — Validate a Binary Search Tree

Difficulty: Medium | Topic: Trees, BSTs & Heaps



Sample input

```
7
8 4 12 2 10 9 14
```

Sample output

```
INVALID
```

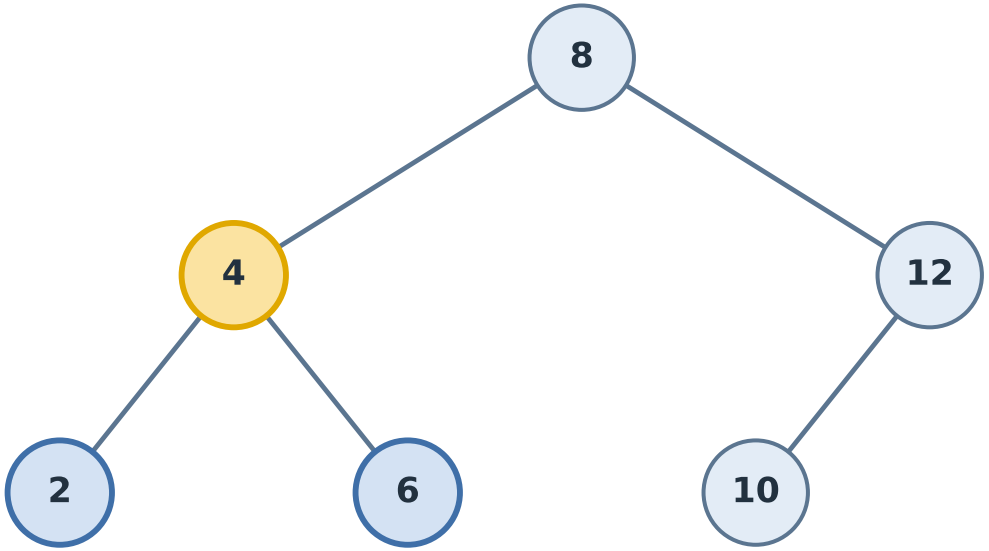
Below each node is the open range its value must lie in, inherited from all its ancestors. Red: a node outside its range.

Walkthrough

Node 10 is a valid right child of 4 (10 > 4), but it is also in the left subtree of 8, so it must be less than 8. 10 is outside (4, 8), so the tree is INVALID.

M18-11 — Find the Lowest Common Ancestor in a BST

Difficulty: Medium | Topic: Trees, BSTs & Heaps



A = 2 and B = 6 lowest common ancestor

Blue: the two query values. Gold: their lowest common ancestor, where their search paths split.

Sample input

```
6 2 6
8 4 12 2 6 10
```

Sample output

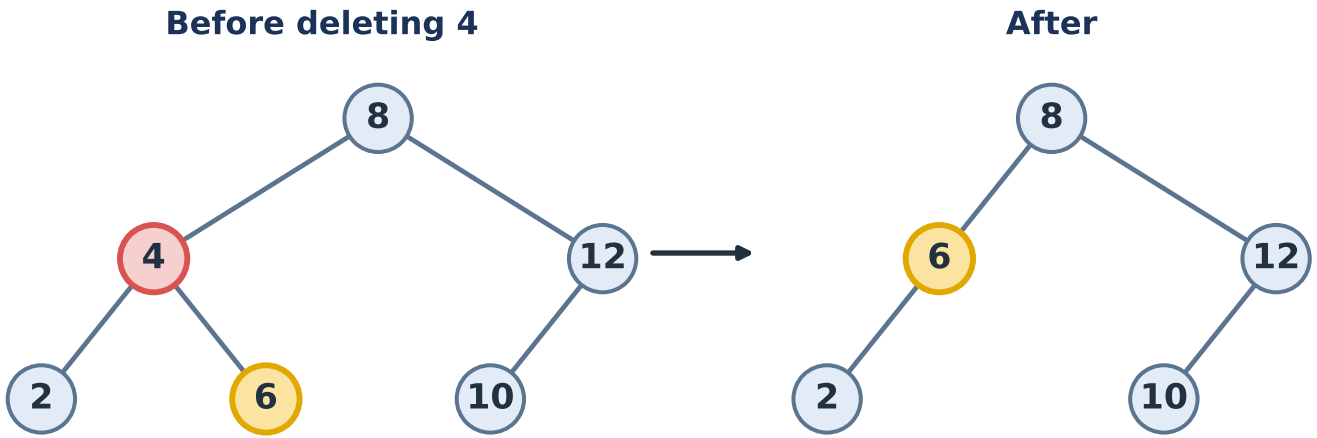
```
4
```

Walkthrough

Both 2 and 6 are smaller than 8, so the LCA is in 8's left subtree. At 4, 2 goes left and 6 goes right, so 4 is the lowest node with both below it.

M18-12 — Delete a Value from a BST

Difficulty: Medium | Topic: Trees, BSTs & Heaps



deleted (two children) inorder successor 6

Left: node 4 (red) has two children, so it is replaced by its inorder successor 6 (gold), the smallest value in its right subtree.

Sample input

```
6 4
8 4 12 2 6 10
```

Sample output

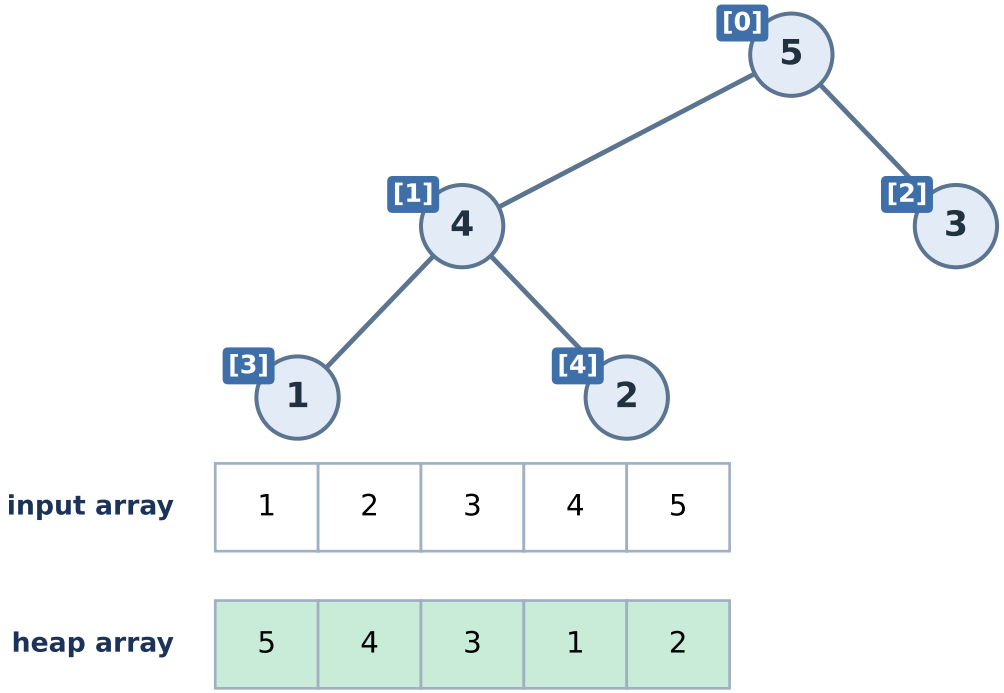
```
8 6 2 12 10
```

Walkthrough

6 moves into 4's position and its original node is removed. The resulting tree is 8 → (6 → 2) and (12 → 10), whose preorder traversal is 8 6 2 12 10.

M18-13 — Build a Max Heap

Difficulty: Medium | Topic: Trees, BSTs & Heaps



Sample input

```
5
1 2 3 4 5
```

Sample output

```
5 4 3 1 2
```

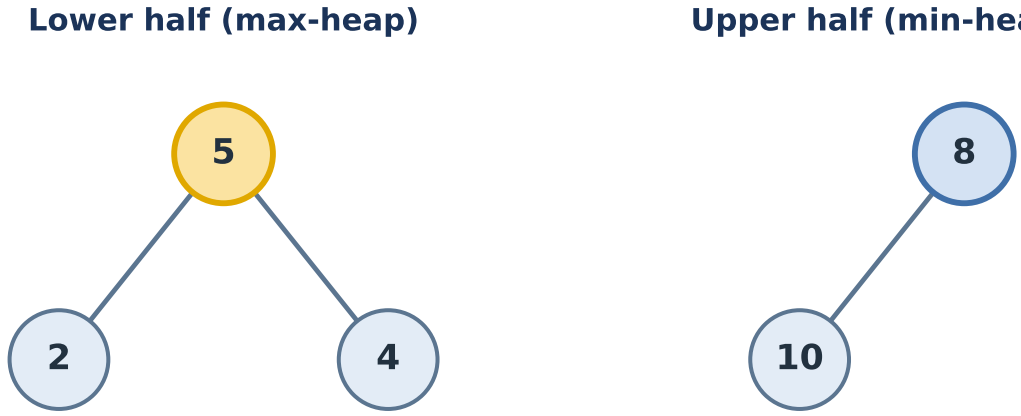
The heap result drawn as a tree; blue badges are array indexes (children of i are 2i + 1 and 2i + 2).

Walkthrough

Sift down index 1 (2 swaps with its larger child 5), then index 0 (1 swaps with 5, then with 4). Result: 5 4 3 1 2. Inserting one at a time would give 5 4 2 1 3 instead.

M18-14 — Find the Running Median of a Data Stream

Difficulty: Medium | Topic: Trees, BSTs & Heaps



After all 5 values: lower has 3 values, upper has 2, so the median is the lower root, 5.0

Sample input

```
5
5 2 10 4 8
```

Sample output

```
5.0
3.5
5.0
4.5
5.0
```

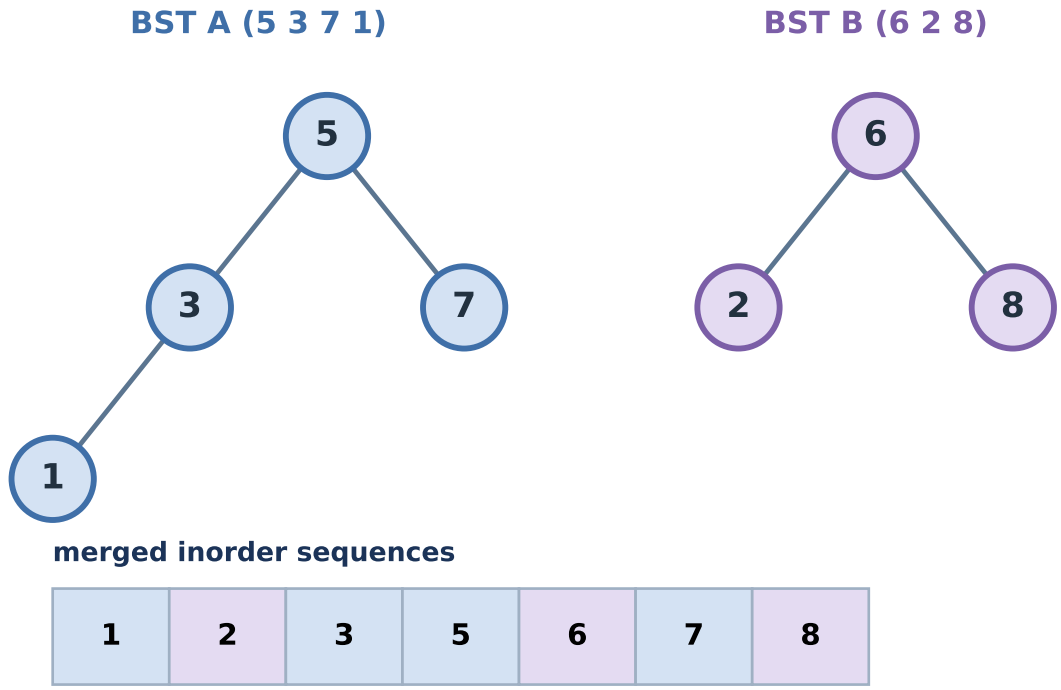
Final state of the two heaps for the stream 5 2 10 4 8. Gold: the lower half's maximum; blue: the upper half's minimum.

Walkthrough

Every value in the lower heap is ≤ every value in the upper heap. With an odd count, the lower heap holds one extra value, so its root (5) is the median; with an even count, the median is the average of both roots.

M18-15 — Merge Two BSTs into a Sorted Sequence

Difficulty: Medium | Topic: Trees, BSTs & Heaps



Sample input

```
4 3
5 3 7 1
6 2 8
```

Sample output

```
1 2 3 5 6 7 8
```

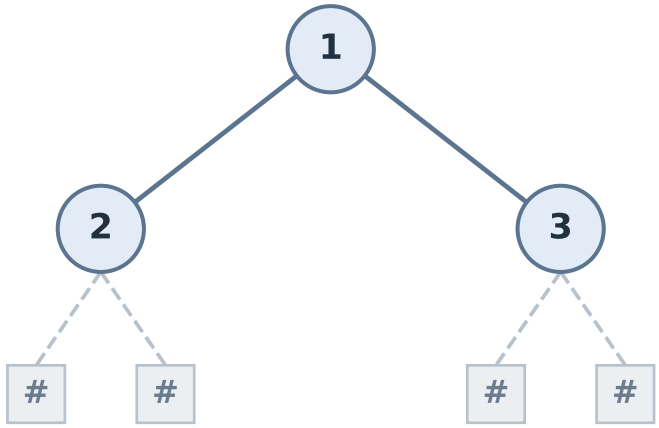
Two separate BSTs (blue and purple) and their merged ascending sequence; each value keeps the color of the tree it came from.

Walkthrough

Inorder traversal gives 1 3 5 7 for A and 2 6 8 for B. Merging the two sorted lists with two pointers gives 1 2 3 5 6 7 8.

M18-16 — Serialize and Deserialize a Binary Tree

Difficulty: Hard | Topic: Trees, BSTs & Heaps



preorder tokens (2 × 3 nodes + 1 = 7)

1	2	#	#	3	#	#

Sample input

```
3
1 2 3
```

Sample output

```
1 2 # # 3 # #
2 1 3
```

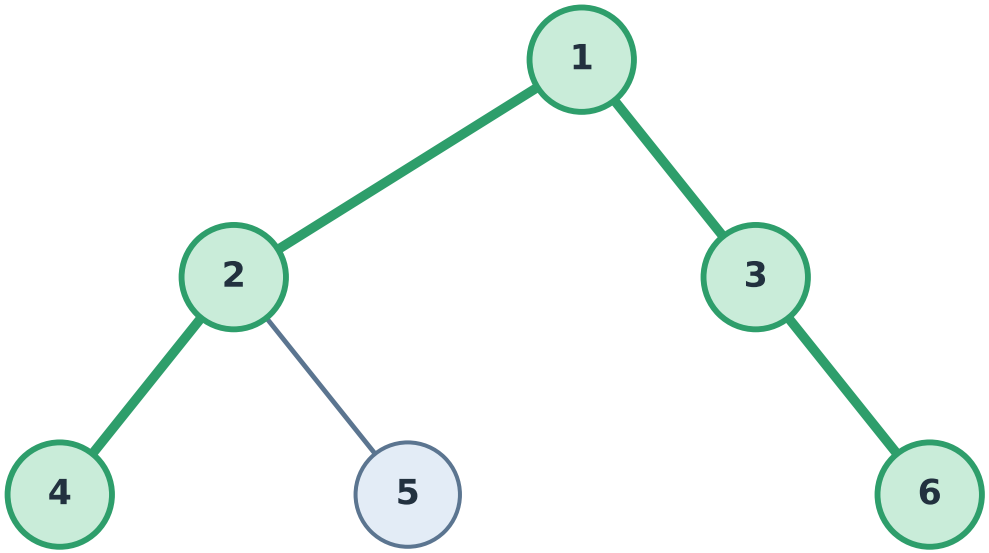
Grey # boxes are the missing children that the serialization records; the token strip shows the preorder output.

Walkthrough

Preorder writes 1, then 2 and its two missing children (# #), then 3 and its two missing children (# #). The # markers let deserialization rebuild exactly the same shape, whose inorder is 2 1 3.

M18-17 — Find the Diameter of a Binary Tree

Difficulty: Hard | Topic: Trees, BSTs & Heaps



Sample input

```
7
1 2 3 4 5 -1 6
```

Sample output

```
4
```

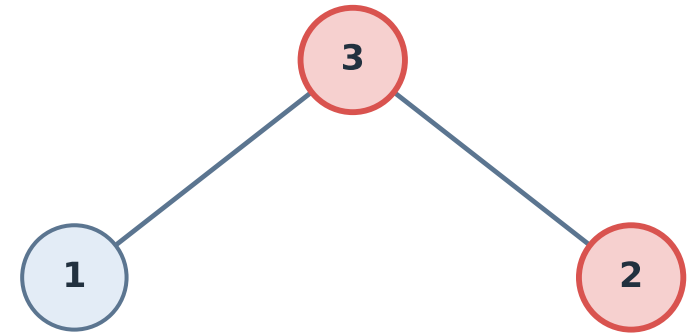
The highlighted path is a longest path between two nodes, measured in edges.

Walkthrough

4 → 2 → 1 → 3 → 6 has 5 nodes and 4 edges. At node 1, the left height (2) plus the right height (2) gives 4, the largest value found in the postorder pass.

M18-18 — Recover a BST with Two Swapped Values

Difficulty: Hard | Topic: Trees, BSTs & Heaps



inorder now: 1 3 2 → swap 3 and 2 → 1 2 3

Sample input

```
3
3 1 2
```

Sample output

```
1 2 3
```

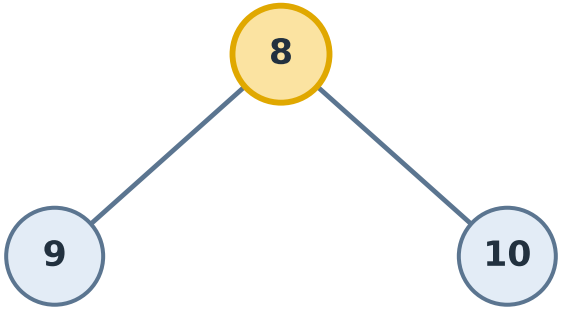
Red: the two nodes whose values were swapped. The structure stays the same; only the two values are exchanged back.

Walkthrough

Inorder of a BST must be increasing. Here it is 1 3 2: the drop from 3 to 2 identifies the misplaced pair, and swapping them gives root 2 with children 1 and 3.

M18-19 — Find the Kth Largest Element in a Data Stream

Difficulty: Hard | Topic: Trees, BSTs & Heaps



after value:	4	5	8	2	10	9
answer:	NONE	NONE	4	4	5	8

Sample input

```
6 3
4 5 8 2 10 9
```

Sample output

```
NONE NONE 4 4 5 8
```

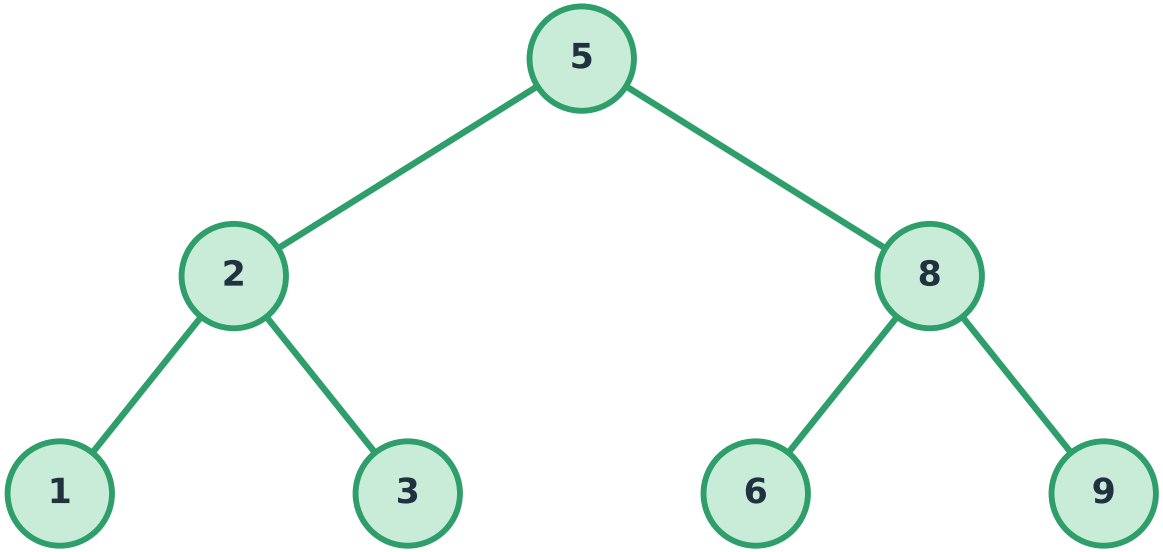
The min-heap after the whole stream holds the 3 largest values; its root (gold) is the 3rd largest.

Walkthrough

Until three values have arrived the answer is NONE. Afterwards the heap keeps only the K largest values, dropping its root whenever it grows past K. After 9 arrives it holds 8, 9, 10, so the answer is 8.

M18-20 — Find the Maximum Sum of a BST Subtree

Difficulty: Hard | Topic: Trees, BSTs & Heaps



sum = 5 + 2 + 8 + 1 + 3 + 6 + 9 = 34

Sample input

```
7
5 2 8 1 3 6 9
```

Sample output

```
34
```

Green: the valid BST subtree with the largest sum. Here the whole tree qualifies.

Walkthrough

Every left subtree holds smaller values and every right subtree larger ones, at every node, so the entire tree is a valid BST. No subtree can exceed the whole tree's sum of 34.